Lecture 21 : Project Estimation Techniques (Contd.)

Introduction

In the dynamic world of software development, accurately estimating the effort, cost, and schedule of a project is paramount for successful delivery and resource management. Without reliable estimation techniques, projects can easily run over budget, miss deadlines, and fail to meet stakeholder expectations. This lecture delves into parametric models, a powerful category of estimation techniques that leverage historical data and mathematical formulas to predict project attributes. Specifically, we will explore the COCOMO (Constructive Cost Model) family, a widely recognized and influential model developed by Dr. Barry Boehm.

COCOMO provides a structured approach to quantify the resources needed for software projects, moving beyond mere guesswork. By understanding its underlying principles, different modes, and mathematical models, aspiring software project managers can gain a crucial skill set for planning and executing projects effectively. This session will focus on the foundational COCOMO 81 basic model, illustrating how to apply its equations to real-world scenarios and make informed decisions about project feasibility and resource allocation.

Core Concepts

Term	Definition	Significance
`COCOMO`	Constructive Cost Model, a parametric model for software cost estimation, first published by Dr. Barry Boehm in 1981.	Provides a structured, data-driven approach to estimate software project effort, cost, and schedule.
`Organic Mode`	A software development environment characterized by familiar, stable conditions, where the product is similar to previously developed ones, typically less than 50,000 Delivered Source Instructions (DSIs).	Represents projects with low complexity and high team familiarity, often leading to lower estimation constants.

Term	Definition	Significance
`Semi-detached Mode`	A development environment falling between organic and embedded modes, involving moderate complexity and some innovation.	Caters to projects with mixed characteristics, requiring a balance between established practices and new approaches.
`Embedded Mode`	A development environment for completely new products requiring significant innovation, inflexible constraints, and stringent interface requirements, such as operating systems or real-time systems.	Represents highly complex and novel projects, demanding greater effort and longer development times due to inherent challenges.
`KLOC`	Thousands of Lines of Code, a common metric for measuring software size in COCOMO models.	Serves as the primary input for size-based estimation formulas, directly influencing effort and schedule calculations.
`Effort`	The total person-months (or man-months) required to complete a software project.	A critical output of COCOMO, used to determine staffing needs, project cost, and overall resource allocation.
`Nominal Development Time`	The estimated duration, in months, required to complete a software project.	Essential for project scheduling and setting realistic deadlines, derived from the calculated effort.
`Basic Model`	The simplest COCOMO model, used for early, rough estimates of project cost, performance, and schedule.	Provides quick, initial estimations with an accuracy of within a factor of 2 of actuals 60% of the time.
`Intermediate Model`	A more refined COCOMO model that incorporates an Effort	Offers improved accuracy (within 20% of actuals 68%

Term	Definition	Significance
	Adjustment Factor (EAF) from 15 cost drivers.	of the time) by considering various project attributes.
`Detailed Model`	The most comprehensive COCOMO model, using different effort multipliers for each phase of a project.	Provides the highest level of estimation granularity, though the Intermediate Model is often preferred by project managers for its balance of detail and practicality.

Detailed Analysis

1. COCOMO Overview and Development Modes

The COCOMO (Constructive Cost Model) is a foundational parametric model for software cost estimation, first introduced by Dr. Barry Boehm in 1981. It was developed based on statistical regression analysis of data from 63 past projects, ranging in size from 2,000 to 512,000 Delivered Source Instructions (DSIs). COCOMO 81, specifically, relies on industry productivity standards and allows organizations to benchmark their software development productivity.

COCOMO categorizes software projects into three distinct development environments or "modes," each reflecting different levels of complexity and familiarity:

- **Organic Mode**: Projects in this mode are developed in a familiar and stable environment. The product is similar to previously developed ones, and the team has extensive experience. Typically, these projects are smaller, often less than 50,000 DSIs. Examples include accounting systems or student information systems.
- **Semi-detached Mode**: This mode lies between organic and embedded. Projects here involve a mix of familiar and new elements, requiring some innovation but not as much as embedded systems. Examples include utility applications like compilers or linkers.
- **Embedded Mode**: These projects involve developing completely new products that require a great deal of innovation, operate under inflexible constraints, and have stringent interface requirements. Examples include operating systems, real-time systems, or complex embedded systems.

The characteristics of these modes influence various project aspects, as summarized below:

Feature	Organic	Semidetached	Embedded
Organizational understanding of product and objectives	Thorough	Considerable	General
Experience in working with related software systems	Extensive	Considerable	Moderate
Need for software conformance with pre-established requirements	Basic	Considerable	Full
Need for software conformance with external interface specifications	Basic	Considerable	Full

2. COCOMO Models: Basic, Intermediate, and Detailed

COCOMO offers three increasingly complex models for estimation:

- **Basic Model**: This is the simplest model, primarily used for early, rough estimates of project cost, performance, and schedule. Its accuracy is generally within a factor of 2 of actuals, 60% of the time. It provides a quick initial assessment.
- **Intermediate Model**: This model enhances accuracy by incorporating an Effort Adjustment Factor (EAF) derived from 15 cost drivers. These drivers account for various project attributes like product complexity, required reliability, and developer capability. While more accurate, it still doesn't account for 10-20% of costs related to training, maintenance, or quality. Its accuracy is within 20% of actuals, 68% of the time.
- **Detailed Model**: Also known as the complete COCOMO, this model uses different effort multipliers for each phase of the project. It offers the most granular estimation but is also the most complex. Most project managers tend to use the Intermediate Model due to its balance of accuracy and practicality.

This lecture primarily focuses on the Basic COCOMO 81 model.

3. Basic COCOMO 81 Equations for Effort and Schedule

The Basic COCOMO 81 model uses two fundamental equations to estimate effort and nominal development time:

a. Effort Equation:
The effort required for a project is calculated as:
`Effort = c * (Size)^k`

Where:
* `Effort` is measured in person-months (PM).
* `c` is a constant that depends on the developmental mode.
* `Size` is measured in KLOC (Thousands of Source Lines of Code).
* `k` is an exponent constant specific to the developmental mode.

The values for `c` and `k` for each mode are:

System Type	c	k
Organic	2.4	1.05
Semi-detached	3.0	1.12
Embedded	3.6	1.20

The `k` exponent, being greater than 1, indicates that effort is somewhat super-linear with problem size. This means larger projects require disproportionately more effort, partly due to increased management overheads.

b. Schedule Equation (Nominal Development Time):
The nominal development time (schedule) for a project is calculated as:
`Nominal Development Time = 2.5 * (Effort)^exponent`

Where:
* `Nominal Development Time` is measured in months.
* `2.5` is a constant for all modes.
* `Effort` is the value calculated from the effort equation.
* `exponent` is a constant that depends on the developmental mode.

The values for the `exponent` for each mode are:

System Type	Exponent
Organic	0.38

System Type	Exponent
Semi-detached	0.35
Embedded	0.32

The exponent for development time is less than 1, indicating that development time is a sub-linear function of product size. This implies that while product size increases, development time does not increase proportionally. For instance, if a product's size doubles, its development time does not necessarily double. This is because larger projects often allow for more parallel activities, which can be carried out simultaneously by a larger team of engineers, thus optimizing the schedule. Interestingly, the time taken for development tends to be roughly similar across all three product categories for a given KLOC (e.g., a 60 KLOC program might take approximately 18 months regardless of whether it's organic, semi-detached, or embedded).

4. Practical Examples of COCOMO 81 Application

Let's illustrate the application of Basic COCOMO 81 with a few examples:

Example 1: Organic Software Product
An organic software product is estimated to be 32 KLOC. The average developer salary is Rs. 15,000 per month.
* **Effort Calculation**: For an organic product, c = 2.4 and k = 1.05.
`Effort = 2.4 * (32)^1.05 = 91 Person-Months (PM)`
* **Nominal Development Time Calculation**: For an organic product, exponent = 0.38.
`Nominal Development Time = 2.5 * (91)^0.38 = 14 months`
* **Cost Calculation**:
`Staff Cost = 91 PM * Rs. 15,000/PM = Rs. 1,365,000`

Example 2: Organic Project with 100 KLOC
An organic product is 100 KLOC.
* **Effort Calculation**: For an organic product, c = 2.4 and k = 1.05.
`Effort = 2.4 * (100)^1.05 = 302.1 Person-Months`
* **Nominal Development Time Calculation**: For an organic product, exponent = 0.38.
`Nominal Development Time = 2.5 * (302.1)^0.38 = 8.6 months`

Example 3: Business Application (Organic Type) with 40 KLOC
A business application (classified as organic) is 40 KLOC. In-house developers cost Rs. 6,000 per programmer-month.

* **Effort Calculation**: For an organic product, c = 2.4 and k = 1.05.
`Effort = 2.4 * (40)^1.05 = 115.5 Person-Months`
* **Cost Calculation**:
`Total Cost = 115.5 PM * Rs. 6,000/PM = Rs. 693,000`

Example 4: Organic Project with 7.5 KLOC
An organic project is 7.5 KLOC.
* **Effort Calculation**: For an organic product, c = 2.4 and k = 1.05.
`Effort = 2.4 * (7.5)^1.05 = 20 Person-Months`
* **Nominal Development Time Calculation**: For an organic product, exponent = 0.38.
`Nominal Development Time = 2.5 * (20)^0.38 = 8 months`
* **Average Staff Required**:
`Average Staff = Effort / Development Time = 20 PM / 8 months = 2.5 staff members`
* **Productivity**:
`Productivity = Total Size (LOC) / Effort (Staff-Months) = 7500 LOC / 20 PM = 375 LOC/Staff-Month`

Example 5: Embedded Project with 50 KLOC
An embedded project is 50 KLOC.
* **Effort Calculation**: For an embedded product, c = 3.6 and k = 1.20.
`Effort = 3.6 * (50)^1.20 = 394 Person-Months`
* **Nominal Development Time Calculation**: For an embedded product, exponent = 0.32.
`Nominal Development Time = 2.5 * (394)^0.32 = 17 months`
* **Average Staff Required**:
`Average Staff = Effort / Development Time = 394 PM / 17 months = 23 staff members`
* **Productivity**:
`Productivity = Total Size (LOC) / Effort (Staff-Months) = 50000 LOC / 394 PM = 127 LOC/Staff-Month`

These examples demonstrate how to systematically calculate effort, development time, cost, staffing, and productivity based on the project's mode and size.

5. The Make or Buy Decision

Project estimation techniques like COCOMO are crucial for making strategic "make or buy" decisions. For instance, if a company needs a software package (e.g., 30 KLOC for customer data mining) and can hire developers at Rs. 50,000 per month, but a commercial off-the-shelf (COTS) solution costs Rs. 100,000, the company must evaluate which option is more viable.

To make this decision, one would:
1. Identify the product type (e.g., customer data mining might be organic).
2. Calculate the in-house development effort and nominal development time using COCOMO.
3. Determine the total in-house development cost (Effort * Developer Salary).
4. Compare this internal cost with the external purchase cost (Rs. 100,000).

Beyond direct cost, other factors influence the make-or-buy decision:
* **Availability**: Will the COTS product be available sooner than internally developed software?
* **Total Cost of Ownership**: Is the cost of acquisition plus customization for COTS less than the internal development cost?
* **Support and Maintenance**: Will the cost of outside support (e.g., maintenance contracts) be less than the cost of internal support?

These considerations help in making a well-informed decision that aligns with business objectives and resource constraints.

Key Takeaways

- [ ] COCOMO (Constructive Cost Model) is a parametric model for estimating software project effort, cost, and schedule.
- [ ] COCOMO 81 was developed by Dr. Barry Boehm in 1981, based on statistical data from past projects.
- [ ] Software projects are categorized into three modes: Organic (simple, familiar), Semi-detached (moderate complexity), and Embedded (complex, innovative).
- [ ] The Basic COCOMO model provides early, rough estimates using size (KLOC) as the primary input.
- [ ] The Effort equation is `Effort = c * (Size)^k`, where `c` and `k` are mode-dependent constants.
- [ ] The Nominal Development Time equation is `Nominal Development Time = 2.5 * (Effort)^exponent`, with the exponent also mode-dependent.
- [ ] Effort is super-linear with problem size, while development time is sub-linear, allowing for parallel activities in larger projects.
- [ ] COCOMO calculations can determine total cost, average staff required, and project productivity.
- [ ] Project estimation is crucial for strategic "make or buy" decisions, considering cost, time, and support.

Conclusion

The COCOMO 81 basic model provides a fundamental yet powerful framework for estimating software project parameters. By understanding the distinctions between organic, semi-detached, and embedded project modes, and applying the corresponding constants in the effort and schedule equations, project managers can generate initial, data-backed estimates. These estimates are not just numbers; they are critical tools for resource allocation, budget planning, and setting realistic timelines.

Furthermore, the ability to calculate metrics like average staff required and productivity per staff-month offers deeper insights into project efficiency. Ultimately, the principles of COCOMO extend beyond mere calculation, empowering project leaders to make informed strategic decisions, such as whether to develop software in-house or acquire a commercial solution, by weighing costs, timelines, and support implications. This foundational knowledge of COCOMO is an indispensable asset for effective software project management.